LADDER LAB — Teacher Guide · the stoplight PLC lab · one page · runs fully offline

THE SCAN CYCLE, IN PLAIN TERMS

A PLC is not event-driven. Forever, it: **(1) reads all inputs** into an *input image table* (a snapshot), **(2) solves the ladder top-to-bottom, left-to-right** using that snapshot, **(3) writes all outputs** at once from the *output image table*. One lap = one **scan** (real PLCs: ~1–20 ms). Pressing a button does *nothing* until the next input read — Ladder Lab's **Step Scan** button and the image tables let students watch this happen, which no physical trainer can show.

WHY RUNG ORDER MATTERS

A rung that writes a bit affects **later rungs in the same scan**, but **earlier rungs only see it next scan**. Load **Program 6 (Scan Order Demo)**, hold START, and single-step the A and B variants: the same two rungs, in opposite order, differ by one full scan. That one-scan difference is the root of real interlock and sequencing bugs.

LATCH (OTL/OTU) VS. SEAL-IN (OTE)

An **OTE** coil is re-solved every scan — true rung, on; false rung, off. A **seal-in** keeps an OTE on by wiring the output's own contact in parallel with START (Program 5): power loss = motor stays off (usually the safe choice). **OTL** stays on until an explicit **OTU** — it "remembers" through a false rung, which is why the crosswalk request in Program 2 uses one.

45-MINUTE LESSON PLAN

Time	Activity
0–5	**Hook:** Trainer mode, Program 1 running. "What is deciding when the lights change?" Introduce the PLC.
5–12	**Scan cycle:** Pause → Step Scan with the watch window open. Press `PED_NS` while paused; step once — *now* it appears in the input image. Show Program 6 A/B for rung order.
12–20	**Read the ladder:** Program 5 (seal-in). Click each instruction to see its plain-English role. Then Program 1: find the 4 step bits, the timers, the interlock contacts.
20–32	**Build:** Challenges → students do (a) seal-in, then (b) delayed start. Auto-grader gives step-by-step feedback. Fast finishers: challenge (c).
32–42	**Troubleshoot:** Hide the ladder pane. Inject a random fault into Program 1 or 2; class predicts the bug from the intersection's behavior alone; reveal the ladder and confirm. Solve times make a friendly leaderboard.
42–45	**Debrief** with discussion questions; assign challenge (d) as extension.

COMMON MISCONCEPTIONS (ADDRESS HEAD-ON)

- **"Ladder is a schematic."** It *looks* like relay wiring but is a **program**: solved in order, one rung at a time, over and over. Electricity flows everywhere at once; a scan does not.
- **"Outputs change mid-rung."** Real outputs update only at the **end of the scan**, all together (watch the output image table while stepping).
- **"Pressing a button acts instantly."** It acts at the **next input scan**. In step mode a press does nothing until you step.
- **"A timer counts wall-clock."** TON accumulates only while its rung is true, in scan-time increments, and resets when the rung goes false.
- **"Two coils with the same tag both work."** The **last rung wins** every scan (see the double-coil fault).

DISCUSSION QUESTIONS

1. Why can't both directions ever be green in Program 1, even if a timer misbehaves? (Two independent defenses: the step sequence *and* the XIO interlock contacts.)
2. Why is the walk request latched instead of wired straight to the walk sign? What would happen if it weren't?
3. In the Scan Order Demo, which variant would you choose for an emergency-stop rung, and why?
4. Seal-in vs. OTL for the motor: which restarts by itself after a power blip? Which is safer, and when?
5. Our simulated scan is 20 ms. What real-world inputs could a PLC miss entirely between scans, and how would you catch them?

CHEAT SHEET

Program	Teaches	Key tags
1 Basic stoplight	timers chained into a 4-step sequence; interlock	`STEP1–4` `NS_GRN` `EW_GRN`
2 + Crosswalk	latched request honored at next safe phase (ONS at phase start)	`PED_NS` `REQ_NS` `WALK_NS`
3 Sensor-actuated	rest state + demand from the `LOOP_EW` car sensor, minimum green	`LOOP_EW`
4 Night flash	selector switch modes; two-timer flasher	`SW_NIGHT` `FLASH`
5 Motor seal-in	start/stop/seal-in — teach before the stoplight	`PB_START` `PB_STOP` `MOTOR`
6 Scan order A/B	rung order = one-scan difference (use Step Scan)	`B_SRC` `B_ECHO`

Settings menu (⚙): presentation mode (big UI + presenter's notes) · hide ladder (predict-then-reveal) · engine self-test · reset everything. The **?** button reopens the how-to.